

Paradigmi di Programmazione - A.A. 2023-24

Esame Scritto del 9/9/2024

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione usando la strategia **call-by-name** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$((\lambda x. \lambda y. xy)((\lambda k. k)y))z$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & ((\lambda x. \lambda y. xy)((\lambda k. k)y))z \\ \equiv_{\alpha} & ((\lambda x. \lambda w. xw)((\lambda k. k)y))z \\ \rightarrow & \underline{(\lambda w. ((\lambda k. k)y)w)}z \\ \rightarrow & \underline{((\lambda k. k)y)}z \\ \rightarrow & yz \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f h x y =  
  if h x then match x with  
    | (0,k) -> k  
    | (_,k) -> h x  
  else y ;;
```

SOLUZIONE:

Struttura del tipo:

```
H -> X -> Y -> RIS
```

Uso per convenzione H, X, Y come variabili di tipo per la variabili h, x, y , RIS come variabile di tipo del risultato, e $A, B, C, \dots$ come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
H = X -> bool      (da guardia dell'if)
X = int * K         (da pattern matching)
RIS = bool          (da secondo caso p.m.)
K = bool            (da casi del p.m.)
Y = RIS             (da ramo else)
```

Ne consegue:

```
H = (int * bool) -> bool
X = (int * bool)
Y = bool
RIS = bool
```

Tipo inferito:

```
((int * bool) -> bool ) -> (int * bool) -> bool -> bool
```

Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive interi "colorati" di nero o di rosso:

```
type colored_int =
| Black of int      (* intero nero *)
| Red of int        (* intero rosso *)
```

si definiscano, usando i costrutti di programmazione funzionale di OCaml, le funzioni `swap_colors` e `seq_len` con tipi

```
swap_colors : colored_int list -> colored_int list
seq_len : colored_int list -> int
```

tali che:

- `swap_colors lis` restituisca una lista analoga a `lis` ma in cui gli interi neri sono rossi e viceversa;
- `seq_len lis` restituisca la lunghezza della più lunga sequenza di interi dello stesso colore consecutivi in `lis`

Ad esempio, data la seguente lista di interi colorati

```
let lis = [Black 10; Red 5; Red 2; Black 4; Black 1; Black 7; Red 2]
```

abbiamo che

```
swap_colors lis;; (* restituisce [Red 10; Black 5; Black 2; Red 4; Red 1; Red 7; Black 2] *)
```

```
seq_len lis;;      (* restituisce 3 -- in quanto ci sono tre interi neri in sequenza *)
```

SOLUZIONE:

Una possibile soluzione:

```
let swap_colors lis =
  let swap elem =
    match elem with
    | Black n -> Red n
    | Red n -> Black n
  in
  List.map swap lis;;

let seq_len lis =
  let f (prev_black,cont,max) elem =
    (match elem with
     | Black n -> if prev_black then
                     if (cont+1) > max then (true,cont+1,cont+1)
                     else (true,cont+1,max)
                   else (true,1,max)
     | Red n -> if not prev_black then
                     if (cont+1) > max then (false,cont+1,cont+1)
                     else (false,cont+1,max)
                   else (false,1,max)
    )
  in match lis with
  | [] -> 0
  | (Black n)::lis' -> let (p,c,m) = List.fold_left f (true,1,1) lis'
                      in m
  | (Red n)::lis' -> let (p,c,m) = List.fold_left f (false,1,1) lis'
                    in m;;
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con un costrutto `Test(f;g;x;y)` che, date due espressioni che descrivono funzioni (non ricorsive) `f` e `g`, un'espressione che descrive un argomento `x` e un'espressione che descrive un possibile risultato `y`, verifichi se l'applicazione di `f` ad `x` dia come risultato `y`. In caso affermativo, il risultato di `Test` sarà `y`, altrimenti il risultato di `Test` dovrà essere ottenuto applicando `g` ad `x`.

NOTA: non usare il costrutto `Apply` nell'implementazione di `Test`.

Alcuni esempi (in una sintassi concreta simile a OCaml):

```
Test ( fun x -> x+1 ; fun x -> x-1 ; 10 ; 11);; (* risultato = 11 *)
Test ( fun x -> x+1 ; fun x -> x-1 ; 10 ; 15);; (* risultato = 9 *)
```

Altro esempio:

```
let f = fun x -> x+1 in
let g = fun x -> x-1 in
let x = 10 in
Test ( f ; g ; x ; x+1);;          (* risultato 11 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.

SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | Test of exp*exp*exp*exp

let rec eval e s = match e with
  ...
  | Test (f,g,x,y) -> (match (eval f s) with
    | Closure (i,body_f,env_f) ->
      let x_val = eval x s in
      let y_val = eval y s in
      let y_res = (eval body_f (bind env_f i x_val)) in
      if y_val = y_res then y_res
      else (match (eval g s) with
        | Closure (j,body_g,env_g) ->
          eval body_g (bind env_g j x_val)
        | _ -> failwith "Error"
      )
    | _ -> failwith "Error"
  )
```